
ASSIGNMENTS

Problem set 3 - A decider for truth in a model for $\mathcal{A}$

AI504 — Knowledge Representation

Simon Holm · Johannes Rothe · Shuagib Ibrahim · Anne Sofie Høj · Daniel Nissen

March, 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1. Problem 1: The type of sentences	3
2. Problem 2: The type of models	4
3. Problem 3: A decider for $\models$	5
4. Appendix	6

1. Problem 1: The type of sentences

Define a unary type constructor `Sentence` following Definition 1.1. If p is a type, then `Sentence p` must be a type representing all possible sentences of logic $\mathcal{A}$ using p as a signature. In particular, if $a :: p$ and $b :: p$, then there should be some expression `AllAre a b :: p`. Only expressions constructed using such a rule should have type `Sentence p`.

Example 1.1: Let `Char` be the set of all Unicode code points. Then:

$$\phi := \text{All } 'p' \text{ are } 'q'$$

is a sentence over signature `Char`.

```
phi :: Sentence Char
phi AllAre 'p' 'q'
```

1.1. Solution

We want to make a type constructor. That is `data typeName type = arguments`

```
data Sentence p = AllAre p p
```

We define `Sentence` taking the type p , and argues that `AllAre p p` (All p are p , where p is just a placeholder). This type constructors ensures that if there exists $a :: p$ and $b :: p$, then

```
AllAre a b :: Sentence p
```

2. Problem 2: The type of models

Define a binary type constructor `Model` following Definition 1.3. If `p` and `m` are types, then `Model p m` must be a type representing all possible models for logic $\mathcal{A}$ with signature `p` and universe `m`. Let `f` be a function whose domain is `p` and whose codomain is the type of all finite2 sets of terms in `m`, then there should be some expression `Mod f :: Model p m`. Only expressions constructed using such a rule should have type `Model p m`. Search the imported library for the “type of finite sets of expressions of a certain type”.

Note that terms of `Model p m` only need to specify the interpretation function $\llbracket \cdot \rrbracket$ (named `f` above) from the pair $\mathcal{M} = (M, \llbracket \cdot \rrbracket)$ as described in Definition 1.3. This is because the universe M is already present in the type as `Model p m` as `m`.

Example 2.1: Let `Bool` and `String` be the set of all boolean values and countable Unicode strings respectively. We use `Bool` as signature. Then:

$$\begin{aligned}\mathcal{M}_0 &:= (\text{String}, \llbracket \cdot \rrbracket_0 : \text{Bool} \rightarrow \mathcal{P}(\text{String})) \\ \forall b \in \text{Bool}. \quad \llbracket b \rrbracket_0 &:= \emptyset \\ \mathcal{M}_1 &= (\mathbb{Z}, \llbracket \cdot \rrbracket_1 : \mathbb{Z} \rightarrow \mathcal{P}(\mathbb{Z})) \\ \forall z \in \mathbb{Z}. \quad \llbracket z \rrbracket_1 &:= \{-|z|, \dots, |z|\}\end{aligned}$$

```
m0 :: Model Bool String
m0 = Mod (const empty)

m1 :: Model Integer Integer
m1 = Mod (\ z -> fromList [- abs z .. abs z])
```

2.1. Solution

We want to make another type constructor much like in Section 1. This time it should define the $\mathcal{M}$ as

$$\mathcal{M} = (P, \llbracket \cdot \rrbracket : P \rightarrow \mathcal{P}(M))$$

For a Haskell implementation we need a constructor that takes both a signature `p` and a universe (domain) `m`. It should then use the `Mod f`, to map the signatures to a some set (usually a subset of the domain)

```
data Model p m = Mod (p -> Set m)
```

3. Problem 3: A decider for $\models$

Let U be some set with a subset $Y \subseteq U$. We can call (U, Y) a decision problem, and every element of U an instance of decision problem U . The elements in Y are yes-instances, i.e. instances for which the correct answer is “Yes!”. The elements in $U \setminus Y$ are no-instances, i.e. instances for which the correct answer is “No!”. A decider for (U, Y) is a computer program which accepts elements of U as inputs. It must terminate and return True when the input is in Y , while it must terminate and return False when the input is not in Y .

Fix a signature P and a univers M . Consider the decision problem (U, Y) where:

$$U := \{(\mathcal{M}, \phi) \mid \mathcal{M} \text{ is a models and } \phi \text{ is a formula}\}$$

$$Y := \{(\mathcal{M}, \phi) \in U \mid \mathcal{M} \models \phi\}$$

Define an infix binary operator $\models$ which works as a decider for this decision problem. This operator is polymorphic in a generic type p representing P , and in a type m implementing the `Ord` type class³ and representing M . The operator should be curried: the two expressions representing $\mathcal{M}$ and ϕ must be passed as the left and right arguments respectively.

Example 3.1: Consider the previous example again, and remember that the Haskell Language Server can evaluate snippets prefixed with `>>>` in comments (unlike in Python, no need to use an interpreter!).

$$\begin{aligned} \mathcal{M}_0 \models \text{All True are False} &\stackrel{\text{def}}{=} \llbracket \text{True} \rrbracket_0 \subseteq \llbracket \text{False} \rrbracket_0 \\ &\stackrel{\text{def}}{=} \emptyset \subseteq \emptyset \\ \mathcal{M}_0 \models \text{All True are False} &\stackrel{\text{def}}{=} \llbracket -3 \rrbracket_1 \subseteq \llbracket 2 \rrbracket_1 \\ &\stackrel{\text{def}}{=} \{-3, \dots, 3\} \subseteq \{-2, \dots, 2\} \end{aligned}$$

The first statement holds, but the second does not.

```
-- >>> m0 |= AllAre True False
-- True
-- >>> m1 |= AllAre (-3) 2
-- False
```

Note that implementing a decider means implementing a function that returns a truth value. This is very different from an implementation of the truth predicate in Definition 1.4, which would return an object representing the mathematical statement $\mathcal{M} \models \phi$, rather than its mere truth value.

3.1. Solution

```
(|=) :: (Ord m) => Model p m -> Sentence p -> Bool
Mod f |= AllAre p q = f p `isSubsetOf` f q
```

Definign that

$$\mathcal{M} \models \text{All } a \text{ are } b \Rightarrow \llbracket a \rrbracket \subseteq \llbracket b \rrbracket$$

4. Appendix

```
import Data.Set

-- Problem 1: A decider for

data Sentence p = AllAre p p

-- Problem 2: The type of models

data Model p m = Mod (p -> Set m)

-- Problem 3: A decider for 'entails'

(|=) :: (Ord m) => Model p m -> Sentence p -> Bool
Mod f |= AllAre p q = f p `isSubsetOf` f q

m0 :: Model Bool String
m0 = Mod (const empty)

m1 :: Model Integer Integer
m1 = Mod (\z -> fromList [-abs z .. abs z])

-- >>> m1 |= AllAre (-3) 2
-- False

-- >>> m0 |= AllAre True False
-- True
```